数据库并发控制：封锁协议、可串行性与两段锁

一、并发控制概述

1. **核心定位**：事务是并发控制的基本单位，并发控制机制的核心是对事务并发操作做**正确调度**，保障**事务隔离性**和**数据库一致性**。
2. **核心问题**：并发操作会破坏数据库一致性，根源是多事务对同一数据的操作调度随机，引发数据异常。

二、并发操作引发的三类数据不一致问题

1. 丢失修改（Lost Update）

- **定义**：两个事务T_1、T_2读入同一数据并修改，T_2的提交结果覆盖T_1的提交结果，导致T_1的修改丢失。
- **实例**：机票售票系统，T_1、T_2均读入机票余额$A=16$，T_1修改$A=15$并写入，T_2仍基于$A=16$修改为$A=15$写入，实际卖出2张票但余额仅减1。
- **符号表示**：$R_1(A)\to R_2(A)\to W_1(A)\to W_2(A)$，$W_1(A)$被$W_2(A)$覆盖。

2. 脏读（Dirty Read）

- **定义**：事务T_1修改数据并写回磁盘，事务T_2读取该修改后的数据，随后T_1因异常**回滚（Rollback）**，数据恢复原值，T_2读取的为**脏数据**。
- **实例**：T_1读$C=100$，修改为$C=200$并写入，T_2读$C=200$，T_1回滚后$C=100$，T_2持有的$C=200$为脏数据。
- **符号表示**：$W_1(C)\to R_2(C)\to \text{Rollback}(T_1)$，$R_2(C)$读取脏数据。

3. 不可重复读（Non-repeatable Read）

- **定义**：事务T_1读取某一数据后，事务T_2对该数据执行更新操作，T_1再次读取同一数据时，结果与第一次不同。
- **实例**：T_1读$A=50$、$B=100$，计算$A+B=150$；T_2修改$B=200$并写入；T_1再次读$A=50$、$B=200$，计算$A+B=250$，结果不一致。
- **符号表示**：$R_1(A)\to R_1(B)\to W_2(B)\to R_1(A)\to R_1(B)$，两次读取$B$结果不同。

注：统一符号规则：$R_i(X)$表示事务T_i读取数据X，$W_i(X)$表示事务T_i写入数据X。

三、并发控制核心技术——封锁（Locking）

1. 封锁的基本概念

事务T对数据对象（表、记录等）操作前，向系统发出加锁请求，加锁后T对该数据对象拥有排他/共享控制权，**直至T释放锁**，其他事务才能对该数据对象执行对应操作，是实现并发控制的核心技术。

2. 两种基本封锁类型

封锁类型	英文名称	加锁事务权限	其他事务权限	核心作用
排他锁（X锁）	Exclusive Lock	读、修改数据X	无法对X加**任何锁**（X/S），直至释放	独占数据，防止并发修改
共享锁（S锁）	Shared Lock	仅读数据X，不可修改	仅可对X加**S锁**，不可加X锁，直至释放	允许多事务并发读，防止修改

3. 封锁相容矩阵（核心）

设已有锁为行，请求锁为列，Y表示相容（可加锁），N表示不相容（不可加锁），$\emptyset$表示无锁：

$$\begin{bmatrix} & \emptyset & X & S \\ \emptyset & Y & Y & Y \\ X & Y & N & N \\ S & Y & N & Y \end{bmatrix} \tag{1}$$

核心结论：X锁与任何锁都不相容，S锁仅与S锁相容。

四、三级封锁协议

封锁协议是**运用X/S锁的规则**，规定：**何时申请锁、持有锁多久、何时释放锁**，协议级别越高，数据一致性保障越强，可解决的不一致问题越多。

1. 一级封锁协议

- **规则**：事务T**修改数据**R**前**，必须对R加$\boldsymbol{X}$锁，**直至事务结束（提交/回滚）**释放。
- **仅读数据无需加锁。**
- **解决问题**：防止**丢失修改**，保证事务可恢复。
- **实例**：机票售票中，T_1修改A前加X锁，T_2请求X锁时等待，T_1修改并释放后，T_2基于最新$A=15$修改，避免丢失。

2. 二级封锁协议

- **规则**：在**一级封锁协议**基础上，增加：事务T**读取数据**R**前**，必须对R加$\boldsymbol{S}$锁，**读完后立即释放**S锁。
- **解决问题**：防止**丢失修改+脏读**，**无法解决不可重复读**（因S锁读完即释，后续数据可被其他事务修改）。
- **实例**：脏读场景中，T_1修改C前加X锁，T_2请求S锁时等待，T_1回滚并释放X锁后，T_2加S锁读$C=100$，避免脏读。

3. 三级封锁协议

- **规则**：在**一级封锁协议**基础上，增加：事务T**读取数据**R**前**，必须对R加$\boldsymbol{S}$锁，**直至事务结束**释放S锁。
- **核心区别**：与二级协议相比，S锁**事务结束才释放**，而非读完即释。
- **解决问题**：防止**丢失修改+脏读+不可重复读**（三级协议是最严格的基础封锁协议）。
- **实例**：不可重复读场景中，T_1读A/B前加S锁，事务结束前不释放，T_2请求X锁修改B时等待，T_1两次读取A/B结果一致。

4. 三级封锁协议对比表

协议级别	加锁规则	解决的不一致问题	未解决的问题
一级	修改加X锁，事务结束释放	丢失修改	脏读、不可重复读
二级	一级+读加S锁，读完释放	丢失修改、脏读	不可重复读
三级	一级+读加S锁，事务结束释放	丢失修改、脏读、不可重复读	无

五、活锁与死锁（封锁技术的衍生问题）

1. 活锁（Live Lock）

- **定义**：多个事务请求封锁同一数据对象，锁释放后，新的事务总是优先获得锁，某一事务**长期等待无法获得锁**，处于“活等”状态。
- **实例**：T_1封锁R，T_2、T_3依次请求封锁R；T_1释放后，T_3获得锁，T_2等待；T_3释放后，T_4获得锁，T_2持续等待。
- **解决方法**：采用**先来先服务（FCFS）**策略，对封锁请求排队，锁释放后，按队列顺序第一个事务获得锁。

2. 死锁（Dead Lock）

- **定义**：两个或多个事务相互封锁对方需要的Data，**彼此等待对方释放锁**，形成循环等待，所有事务均无法继续执行。
- **实例**：T_1封锁R_1并请求封锁R_2，T_2封锁R_2并请求封锁R_1，T_1等待T_2释放R_2，T_2等待T_1释放R_1，形成死锁。
- **解决方法**：分为**预防**和**诊断与解除**两类。

（1）死锁预防（破坏死锁产生的条件）

- **一次封锁法**：要求事务**一次性对所有需要的数据对象加锁**，若有一个加锁失败，全部释放已加锁对象，重新申请。
  - 缺点：扩大封锁范围，**降低系统并发性**；事务执行前难以确定所有需要的数据对象。
- **顺序封锁法**：预先对所有数据对象规定**固定的封锁顺序**，事务必须按该顺序加锁。
  - 缺点：数据库数据对象繁多，**维护封锁顺序成本高**；事务加锁请求动态生成，难以按固定顺序执行。

（2）死锁的诊断与解除

- **超时法**：设置事务等待锁的**时间上限**，若等待时间超过上限，判定为死锁，撤销该事务并释放其锁。
  - 缺点：易**误判死锁**（长事务正常等待被判定为死锁）；上限设置过久，死锁解除延迟。
- **事务等待图法**：用**有向图**$G=(T,U)$动态反映事务等待关系，其中T为事务节点，U为等待边（$T_1 \to T_2$表示T_1等待T_2释放锁）。
  - 诊断：若等待图中**存在回路**，则判定为死锁。
  - 解除：**撤销代价最小的事务**（如执行时间最短、修改数据最少），释放其所有锁，打破回路；若仍有回路，重复操作。

六、并发调度的可串行性

1. 基本概念

- **串行调度**：多个事务按**先后顺序依次执行**，无操作交叉，串行调度一定是正确的（无数据不一致）。
- **并发调度**：多个事务的操作**交叉执行**，调度结果可能正确，也可能错误。
- **可串行化调度**：若一个并发调度的**执行结果**与**某一串行调度**的执行结果完全相同，则称该并发调度为可串行化调度，是**并发事务正确调度的唯一准则**。

2. 冲突可串行化（实际系统主流采用）

（1）冲突操作

指**不同事务**对**同一数据**的**读写操作（R-W）**和**写写操作（W-W）**，冲突操作的执行顺序不可交换，交换后会改变调度结果。

- 冲突操作：$R_i(X)$与$W_j(X)$、$W_i(X)$与$W_j(X)$（$i \neq j$）。
- 非冲突操作：$R_i(X)$与$R_j(X)$、$R_i(X)$与$W_j(Y)$（$X \neq Y$），执行顺序可交换，不影响调度结果。

（2）冲突可串行化定义

在一个并发调度中，**保持冲突操作的执行顺序不变**，通过交换**非冲突操作**的顺序，若能转化为**串行调度**，则称该调度为**冲突可串行化调度**。

（3）核心结论

- 冲突可串行化调度 **一定是** 可串行化调度（充分条件）；
- 可串行化调度 **不一定是** 冲突可串行化调度（非必要条件）。

七、两段锁协议（2PL，Two-Phase Locking）

1. 核心定义

所有事务对数据项的加锁、解锁操作必须分为**两个阶段**执行，是实现**并发调度可串行性**的核心协议，被主流数据库（MySQL、Oracle）采用。

（1）第一阶段：拓展阶段（Growing Phase）

- 仅**申请加锁**（可加X锁/S锁），**不释放任何锁**；
- 直至事务申请到所有需要的锁，进入第二阶段。

（2）第二阶段：收缩阶段（Shrinking Phase）

- 仅**释放锁**（可释放任意已加锁），**不申请任何新锁**；
- 锁释放后，只能执行释放操作，直至事务结束。

2. 合规/违规示例

- **合规**：$S\backslash\text{lock}(A) \rightarrow X\backslash\text{lock}(B) \rightarrow \backslash\text{unlock}(A) \rightarrow \backslash\text{unlock}(B)$（先全加锁，后全释放）；
- **违规**：$S\backslash\text{lock}(A) \rightarrow \backslash\text{unlock}(A) \rightarrow X\backslash\text{lock}(B)$（拓展阶段释放锁，后又申请新锁）。

3. 核心结论

- 若**所有事务都遵循两段锁协议**，则其并发调度**一定是冲突可串行化**的，保证调度正确性；
- 反之，冲突可串行化调度的事务**不一定**都遵循两段锁协议；
- 两段锁协议**不能避免死锁**（事务按不同顺序加锁仍可能形成循环等待）。